

插件编程模型

目录

一、 插件概述	4
1. 插件是什么?	4
2. 为什么要开发插件?	4
3. 插件如何工作?	4
二、 插件开发步骤	5
步骤一：确定应用场景，选择插件基类	5
步骤二：确定事件源与控件	6
步骤三：响应插件事件，实现特定的业务逻辑	7
步骤四：注册插件，绑定到页面	9
三、 插件模型概述	10
视图模型、控件编程模型	10
表单数据模型	11
主实体模型	11
数据包	13
视图模型	14
(1) 表单视图模型	14
(2) 单据视图模型	17
(3) 列表视图模型	18
数据模型	20
(1) 表单数据模型	20
(2) 单据数据模型	22

(3) 列表数据模型	23
四、 案例演示	26
案例 1：主实体模型	26
案例 2：读取 DynamicObject 对象中存储的全部属性的值	27

一、插件概述

1. 插件是什么？

插件就是金蝶云苍穹平台开发的代码文件。插件分为 **Java 插件**和 KS 脚本插件。

2. 为什么要开发插件？

使用苍穹应用开发平台的设计器开发业务对象，全程配置，简单易学，但不够灵活。

配置业务对象时使用的业务语意，需要预先定义，遇到未考虑的业务场景，则无法处理。

业务对象设计器，目标是实现常见的 80%业务语意，利用开放的插件开发，实现剩余 20%的功能。

3. 插件如何工作？

插件可以在**适当的**时机，根据接收到的上下文信息，对系统功能进行控制。

适当的时机，是指插件只会在系统功能运行到了特定时刻，才能收到系统通知，进行功能处理；并不能对系统功能的全过程进行干预。

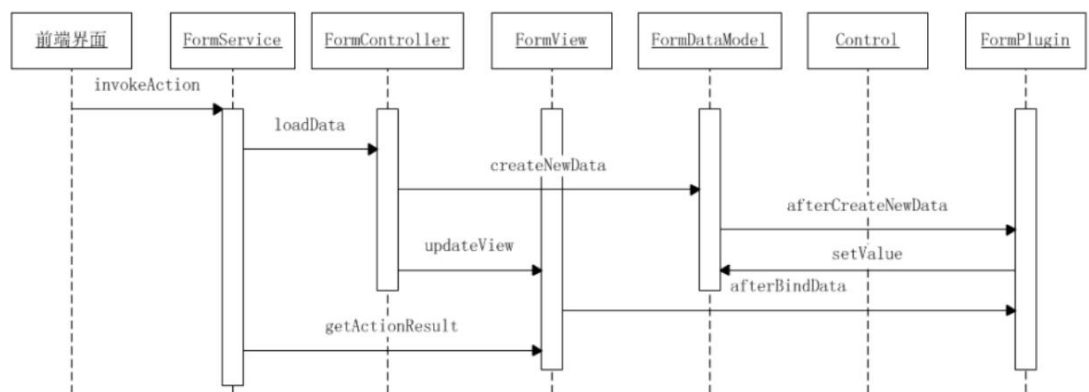
插件拿到的上下文信息，能够调用的控制方法，也是经过系统封装，有所限制；

简单的说，插件可以在系统约束的框架之内，对系统进行适度的干预，在灵活性

与安全性之间有个平衡。

如下图，表单界面在加载、展示期间，关键功能执行时，调用插件接口方法，触发插件事件，通知插件工作；

而插件能够在被调用的事件方法中，调用**表单视图**、控件模型、表单数据模型，对表单界面进行控制。



二、插件开发步骤

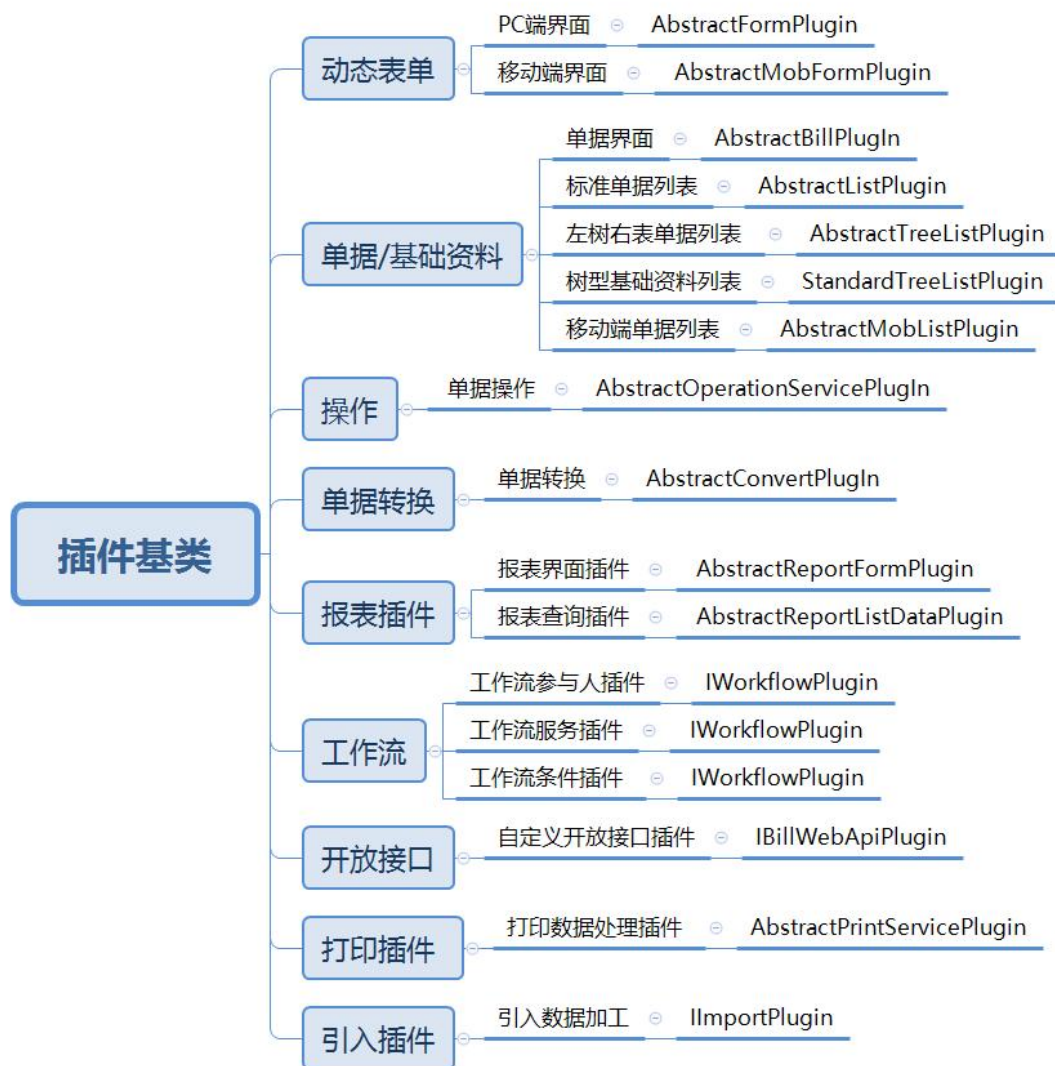
步骤一：确定应用场景，选择插件基类

步骤二：确定事件源与控件

步骤三：响应插件事件，实现特定的业务逻辑

步骤四：注册插件，绑定到页面

步骤一：确定应用场景，选择插件基类



不同的业务对象类型，提供特定的业务功能，有适用的应用场景。

系统为各种业务对象类型、各种应用场景，封装了相应的插件接口、事件方法，定义了抽象插件基类，实现最基本的插件接口。

进行业务功能设计时，首先需要根据业务需求特点，分析业务应用场景，选择业务对象类型；

可以从右图中选择对应插件基类进行扩展

步骤二：确定事件源与控件

有交互界面的应用场景（如表单、单据列表等），在界面加载、关闭时，会触发

相应的插件事件；

另外，用户与界面，以及界面上的控件交互时，也会触发插件事件。

各种控件有自己的功能特点，适用不同的业务需求，提供相应的插件事件；

在进行功能设计时，需要根据业务需求，选用合适的控件，确定需要重写的插件接口方法，响应插件事件。

特别说明：

没有交互界面的应用场景（如单据操作、单据转换、关联反写、生产凭证等），不会与用户发生交互，其插件事件是由服务引擎按顺序触发的。

这些应用场景，不需要关注事件源、控件；只需要根据业务需求，捕获合适的插件事件即可。

步骤三：响应插件事件，实现特定的业务逻辑

运用上下文，开发插件代码，实现特定业务逻辑利用插件基类提供的上下文对象、事件参数、封装好的服务，实现特定的业务逻辑

系统封装了各种插件事件接口；

表单、控件、字段等事件源，各自有选择的支持了部分插件事件接口。

系统会在适当的时机，调用插件事件接口的方法，传入上下文参数，触发插件事件。

比如用户与前端界面、控件交互时，系统会把交互请求传递给表单、控件；表单、控件调用其下的插件事件接口实例（即插件）的方法，触发插件事件。

不同的插件事件，触发的时机、传入的上下文参数、可以控制的功能，差异非常

大。

因此，必须根据业务需求，选择合适的插件事件响应。

无交互界面的场景

没有交互界面的应用场景，插件基类已经实现了必要的插件接口，插件只需要扩展插件基类，重写事件方法即可完成事件的捕捉：

```
public class WriteBackRuleOpPlug extends AbstractOperationServicePlugIn {  
    @Override  
    public void beforeExecuteOperationTransaction(BeforeOperationArgs e) {  
        // TODO : 在此添加业务逻辑  
    }  
}
```

说明：插件基类 AbstractOperationServicePlugIn 已经实现业务操作插件接口，插件直接重写需要响应的事件方法即可。

有交互界面的场景

有交互界面的应用场景，插件基类实现了表单界面支持的插件接口，但没有实现各种控件的插件接口。

如果只是要捕获表单界面事件，则扩展插件基类，重写表单事件方法即可。

如果要捕获各种控件事件，则需要：

在插件类定义，实现控件支持的插件事件接口,例如树形控件支持的节点勾选插件事件接口 TreeNodeCheckListener;

实现控件的插件事件接口中的方法，完成事件的捕捉；

在表单界面插件 registerListener 事件，向控件实例注册本插件实例，完成控件与插件实例的绑定：控件事件发生时，即触发其所绑定的插件事件；


```

public class TreeViewTreeNodeCheck extends AbstractFormPlugin implements TreeNodeCheckListener {
    3 usages

    private final static String KEY_TREEVIEW1 = "treeviewap1";

    @Override
    public void registerListener(EventObject e) {
        super.registerListener(e);
        // 侦听树节点勾选事件
        TreeView treeView = this.getView().getControl(KEY_TREEVIEW1);
        treeView.addTreeNodeCheckListener(this);
    }

    @Override
    public void beforeBindData(EventObject e) {
        super.beforeBindData(e);
        TreeView treeView = this.getView().getControl(KEY_TREEVIEW1);
        treeView.setMulti(true); // 支持多选
    }

    @Override
    public void treeNodeCheck(TreeNodeCheckEvent arg0) {
        TreeView treeView = (TreeView) arg0.getSource();
        if (StringUtils.equals(treeView.getKey(), KEY_TREEVIEW1)){
            List<String> selectNodeIds = treeView.getTreeState().getCheckedNodeIds();
            // TODO 在此添加业务逻辑
        }
    }
}

```

如图，插件需要响应树形控件的节点

勾选 treeNodeCheck 事件

插件基类 AbstractFormPlugin

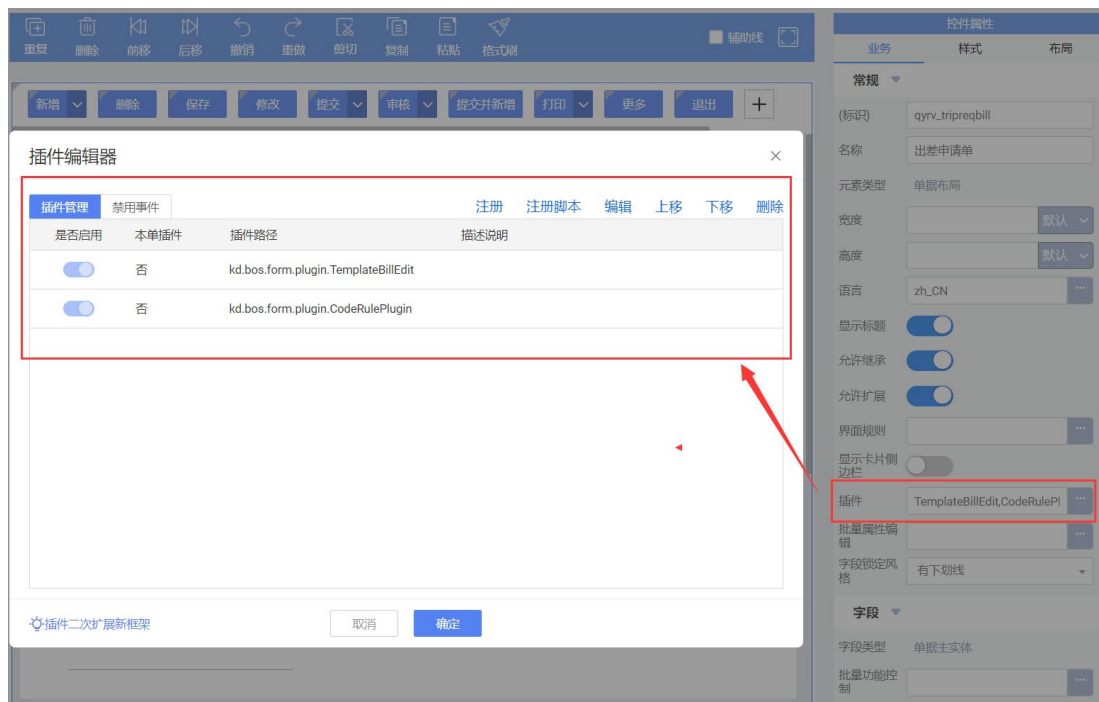
没有实现树形控件的节点点击事件

接口 TreeNodeCheckListener,

插件需要自行实现

步骤四：注册插件，绑定到页面

不同的插件注册的位置稍微有所不同，后面章节将具体插件的时候再详解



三、插件模型概述

视图模型、控件编程模型

金蝶云苍穹是基于 B/S 结构的，使用不同的客户端（PC 端、移动端），通过网络连接到统一的服务端。

用户看到的交互界面，是运行在**客户端**的，而业务逻辑和业务插件，则运行在**服务端**；

业务插件运行在服务端，没办法直接获取到客户端界面上控件句柄，不能直接控制前端控件。

但插件可以通过系统封装的视图模型接口 **IFormView**，间接的访问、控件前端界面；

通过系统封装的各种控件代理对象，间接的访问、控制前端界面上的控件；

这些运行在服务端的控件代理对象，称为控件编程模型，或简称为控件、编程模型等。

可以通过 **this.getView()**方法，获取表单的视图模型接口实例；

可以通过 **this.getView().getControl(String key)**方法，获取控件编程模型实例；

表单数据模型

表单的界面与数据是分离的：

界面显示在客户端浏览器或者移动端，在服务端，系统封装了视图模型及控件编程模型来间接控制前端界面及前端控件；

数据存储在服务端，在服务端，系统封装了数据模型来控制界面上的数据；

表单的数据模型，提供各种方法访问界面数据，并且持有：

主实体模型：MainEntityType，表单运行时元数据对象，包含：

子实体：EntityType，对应单据体、子单据体等；

属性：DynamicProperty，对应字段；

界面数据包：DynamicObject，基于主实体模型构建一个数据字典，存储单据体、字段值；

主实体模型

API 地址：MainEntityType (SDK) (kingdee.com)

动态表单设计完毕，系统会根据表单字段结构，为表单创建一个运行时元数据对象，又称为

表单主实体模型 MainEntityType：

表单上的单据体、字段，都会转为属性对象 DynamicProperty，分为如下三类：

(1) 简单值属性：SimpleProperty，对应普通字段；

(2) 复杂值属性：ComplexProperty，对应基础资料字段，关联基础资料主实体

RefEntityType，嵌套包含基础资料属性；

(3) 集合值属性：CollectionProperty，对应单据体等分录，关联到分录子实体 EntryType，

嵌套包含分属属性；

各属性对象之间，会按照表单设计，具有层级从属关系；

复杂值、集合值属性对象，会关联子实体，继续包含下层属性对象（字段、子单据体等）

主实体模型会被缓存，请勿在插件中直接修改主实体模型内容，以免串账；业务需求必须动态修改主实体模型时，只能修改复制品；

主实体模型 MainEntityType 部分常用方法如下：

MainEntityType 的超类 EntityType 提供的常用方法如下：

方法	说明
getAllEntities	全部子实体
getAllFields	全部字段，不包括系统自动注册的属性对象，如主键
getMainOrg	主业务组织标识，可能为 null
getAppId	业务应用 Id
getPermissionControlType	功能权限控制配置

方法	说明
findProperty	查找属性，会遍历本实体以及子实体

EntityType 的超类 DynamicType 提供的常用方法如下：

方法	说明
getName	实体的标识
getDisplayName	实体的标题

getDBRouteKey	分库标识
getPrimaryKey	主键;动态表单，与物理表格无关，没有主键
getProperties	本实体的属性对象集合，不包括子实体的属性
getAlias	物理表格名称动态表单不与物理表格关联，此属性为空；单据、基础资料的主实体，此属性存储单据表格名
isDbIgnore	是否关联物理表格
registerSimpleProperty	注册新的简单值属性，如文本、数值、日期等
registerComplexProperty	注册新的复杂值属性，如基础资料
registerCollectionProperty	注册新的集合值属性，如单据体
createInstance	基于本实体模型，创建空白的数据包 (DynamicObject)

数据包

表单数据包，DynamicObject 类型，实际是个数据字典：严格按照表单主实体模型的结构，构建数据字典，存储各个属性的值。

表单主实体模型有三种属性类型，在数据包中，有对应的属性值类型：

- (1) 简单值：存储文本、数值、日期、布尔值；
- (2) 复杂值：存储引用的基础资料数据包，也是 DynamicObject 类型；

(3) 集合值：存储数据包集合，DynamicObjectCollection 类型，即 DynamicObject 类型集合；

视图模型

(1) 表单视图模型

动态表单界面插件，可以通过系统封装的表单视图模型，访问界面信息，对界面进行控制。

接口与实现类

动态表单的视图模型接口为 IFormView，定义了各种界面控制方法：

```
package kd.bos.form;

public interface IFormView {.....}
```

动态表单的视图模型实现类为 FormView，实现了接口 IFormView：

```
package kd.bos.mvc.form;

public class FormView extends AbstractFormView {.....}

package kd.bos.form;

public abstract class AbstractFormView implements IFormView {.....}
```

插件可以使用如下代码，获取动态表单界面视图模型实例：

```
FormView view = this.getView();
```

功能方法及使用

动态表单界面视图模型 IFormView 接口，定义了很多方法，下表列出插件需要用到的部分方法

方法	说明
----	----

getPageld	表单被加载时，会随机生成一个界面 Pageld；一个表单被两个用户同时打开时，生成的界面 Pageld 不同；可以根据返回的 pageld，获取到表单视图模型实例；
getView	指定 Pageld，获取对应的表单视图模型；可以据此对目标表单进行控制；
getEntityId	获取表单对应的主实体标识
getModel	获取表单数据模型实例
sendFormAction	把目标表单的控制指令发送给前端；插件调用了其他表单的控制方法后，必须调用本方法，把控制指令，发送给前端；
getParentView	获取父表单视图模型实例；
getMainView	获取主界面视图模型实例；
updateView	把数据模型中的数据，发送到前端界面；定义了多个重载函数，可以指定只刷新单个数据体、单个控件
getControl	获取表单的控件实例
getRootControl	获取表单实例
getService	获取服务实例
invokeOperation	执行操作
activate	激活表单
close	关闭表单

setEnabled	设置控件可用性
setVisible	设置控件可见性
showForm	传入表单显示参数，打开一个新的表单，作为本表单的子表单；该表单关闭时，会触发本表单插件 closedCallBack 事件,请参阅表单 closedCallBack 事件说明及示例；
getFormShowParameter	获取表单显示参数
cacheFormShowParameter	修改表单显示参数对象属性值之后，调用本方法把参数更新到缓存
returnDataToParent	设置返回到父表单的返回值
openUrl	打开一个新窗口链接到指定的 URL
showUpload	显示一个文件上传界面;文件上传完毕，确认返回时，会触发插件 afterUpload 事件;请参阅按钮 afterUpload 事件说明及示例；
showMessage	单据内悬浮消息框，默认没有按钮，自动消失
showErrorMessage	显示错误消息
showOperationResult	显示操作结果
showConfirm	显示确认消息；用户确认完毕，会触发 confirmCallBack 事件；请参阅 confirmCallBack 事件说明及实例
showSuccessNotification	单据内成功悬浮消息框，默认 2 秒自动消失

	消息内容,不能超过 50 字,超过部分用三个点代替
showErrorNotification	单据内失败悬浮消息框, 需要手动关闭,消息内容,不能超过 50 字,超过部分用三个点代替
showTipNotification	单据内提示类别悬浮消息框, 提示类会显示按钮, 需要手动关闭消息内容,不能超过 50 字,超过部分用三个点代替
showRobotMessage	发送消息给机器人助手
closeRobotMessage	关闭消息给机器人助手
showFieldTip	字段上显示提示信息
showFieldTips	字段上显示提示信息 (批量)

(2) 单据视图模型

接口与实现类

单据视图模型接口为 IBillView, 派生自动态表单视图模型接口 IFormView, 增加了少量仅用于单据界面的控制方法:

```
package kd.bos.bill;
```

```
public interface IBillView extends IFormView {.....}
```

单据视图模型实现类为 BillView, 派生自动态表单视图模型实现类 FormView, 实现了单据视图模型接口 IBillView, 并重写了部分动态表单视图模型的方法:

```
package kd.bos.mvc.bill;
```

```
public class BillView extends FormView implements IBillView,
```

IConfirmCallBack {

单据插件，可以直接把插件 getView()方法返回的实例，转为 IBillView 的实例：

IBillView billView = (IBillView)this.getView();

功能方法及使用

单据视图模型接口 IBillView，派生自动态表单视图模型接口 IFormView，建议先阅读动态表单章节，了解 IFormView 接口提供的方法。

IBillView 新增加了如下方法：

方法	说明
setBillStatus	设置单据界面编辑状态：新增、修改、查看、提交、审核等；动态表单界面，没有这些状态的划分
updateViewStatus	根据界面编辑状态，刷新控件、字段的可见性、锁定性

(3) 列表视图模型

接口与实现类

插件可以通过列表视图模型控制列表界面。

单据列表界面，提供了两个层次的视图模型：

- (1) 表单视图模型 IFormView：据此访问表单各种信息；
- (2) 列表视图模型 IListView：据此访问单据列表控件的各种信息；

列表视图模型 IListView，派生自表单视图模型 IFormView，增加了列表访问方法：

package kd.bos.list;

public interface IListView extends IFormView {.....}

在列表界面插件(AbstractListPlugin)中，通过 this.getView()，返回的是 ListView 实例，该实例即可以转为 IFormView 接口，也可以转为 IListView 接口。

```
public class ListViewSample extends AbstractListPlugin {
    private void getListView() {
        IFormView formview = this.getView();
        IListView listview = (IListView) this.getView();
    }
}
```

功能方法及使用

列表视图模型接口 IListView，在动态表单视图模型接口 IFormView 基础上，新增加如下方法：

方法	说明
getBillFormId	获取列表界面绑定的单据
setBillFormId	设置列表界面绑定的单据内部方法，插件请勿调用
getListModel	获取列表数据模型
getFocusRow	获取当前焦点行号，整数
getFocusRowPkId	获取当前焦点行显示的单据数据内码
getCurrentSelectedRowInfo	获取当前焦点行数据详情，明细到单据体行、子单据体行
getSelectedRows	获取当前勾选了的全部行数据详情
getCurrentListAllRowCollection	获取列表全部行数据详情
clearSelection	清除所选的行

returnLookupData	把当前所选行，返回给父界面
refresh	刷新列表，重新取数
getSelectedMainOrgIds	获取选中的主业务组织
setSelectedMainOrgIds	设置选中的主业务组织
export	引出
importData	引入
getTreeListView	获取树形列表视图模型 ITreeListView 仅适用于左树右表列表模式
focusRootNode	树形列表，切换到根节点

数据模型

(1) 表单数据模型

接口与实现类

动态表单界面插件可以通过系统封装的表单数据模型，访问界面数据。

动态表单界面数据模型接口为 IDataModel，提供了各种控制表单数据的方法：

```
package kd.bos.entity.datamodel;

public interface IDataModel extends ISupportInitialize, IEntryOperate,
IDataProvider {.....}
```

动态表单的数据模型实现类为 FormDataModel，实现了接口 IDataModel 的方法：

```
package kd.bos.mvc.form;

public class FormDataModel extends AbstractFormDataModel{

package kd.bos.entity.datamodel;
```

```
public abstract class AbstractFormDataModel implements IDataModel,
```

```
    IRefrencedataProvider {
```

插件，可以通过如下代码，获取到动态表单界面的数据模型实例：

```
IDataModel model = this.getModel();
```

功能方法及使用

动态表单数据模型接口 IDataModel, 定义了很多方法, 下表列出插件需要用到的部分方法：

方法	说明
getDataEntityType	获取运行时表单实体元数据对象，又称为主实体模型；通过表单主实体模型，可以或者界面上包含了那些单据体、字段
getProperty	获取运行时字段元数据对象，又称为实体的属性对象
createNewData	根据表单主实体模型，创建表单新的数据包，字段填写好默认值
getDataEntity	获取表单数据包
updateCache	提交当前表单数据包到缓存
getValue	获取字段值
setValue	设置字段值
setItemValueByNumber	根据基础资料的编码，设置基础资料字段值
setItemValueByID	根据基础资料的内码，设置基础资料字段值
getContextVariable	获取上下文变量
putContextVariable	添加上下文变量

removeContextVariable	删除上下文变量
-----------------------	---------

(2) 单据数据模型

接口与实现类

单据界面数据模型接口为 IBillModel，派生自动态表单界面数据模型接口 IDataModel，增加了单据特用的数据模型控制方法：

```
package kd.bos.entity.datamodel;  
  
public interface IBillModel extends IDataModel{.....}
```

单据界面数据模型接口为 IBillModel，派生自动态表单界面数据模型接口 IDataModel，增加了单据特用的数据模型控制方法：

```
package kd.bos.entity.datamodel;  
  
public interface IBillModel extends IDataModel{.....}
```

单据数据模型实现类为 BillModel，派生自动态表单数据模型实现~~~类

FormDataModel，新增了单据模型接口 IBillModel：

```
package kd.bos.mvc.bill;  
  
public class BillModel extends FormDataModel implements IBillModel {.....}
```

单据插件，可以直接把插件 getModel()方法返回的实例，转为 IBillModel 的实例：

```
IBillModel billModel = (IBillModel)this.getModel();
```

功能方法及使用

单据数据模型接口 IBillModel，派生自动态表单数据模型接口 IDataModel，新增加单据数据控制方法：

方法	说明
getPKValue	获取当前单据主键
setPKValue	设置当前单据的主键
load	指定主键，加载单据
push	传入下推生成的单据

主实体模型 BillEntityType

单据的主实体模型是 BillEntityType，派生自动态表单主实体模型 MainEntityType，增加了单据特别属性：

```
package kd.bos.entity;
```

```
public class BillEntityType extends MainEntityType {.....}
```

新增加了如下方法：

方法	说明
getBillNo	单据编号字段标识
getMainOrgProperty	主业务组织
getBillStatus	数据状态字段标识
getBillType	单据类型字段标识
getForbidStatus	禁用状态字段标识（专用于基础资料）
getMobFormId	绑定的移动单据界面标识
getBillParameter	自定义的单据参数界面标识

(3) 列表数据模型

接口与实现类

单据列表界面，包含了两个层次的数据模型：

(1) 表单数据模型 IDataModel：据此访问列表控件所在的表单数据：列表界面，除了单据列表控件，还可以增加各种自定义字段，需要通过表单数据模型 IDataModel，访问这些自定义字段的值；

(2) 列表数据模型 IListModel：据此访问单据列表控件中的数据；

列表数据模型 IListModel，定义如下：

```
package kd.bos.entity.datamodel;

public interface IListModel {.....}
```

表单数据模型 IDataModel 与列表数据模型 IListModel，分别有各自的实现类及实例，没有派生关系，不能互相转换。

接口与实现类

单据列表插件(AbstractListPlugin)，可以如下代码，分别获取表单数据模型 IDataModel、列表数据模型 IListModel 的实例：

```
public class ListModelSample extends AbstractListPlugin {
    private void getListModel(){
        IDataModel formmodel = this.getModel();
        IListModel listmodel = ((IListView)this.getView()).getListModel();
    }
}
```

功能方法及使用

IListModel 提供的方法如下：

方法	说明
----	----

getDataEntityType	获取数据实体
setDataEntityType	设置数据实体
setPageld	设置 pageld
getData	获取数据
getQingData	获取轻分析数据
setEntityId	设置实体 ID
getEntityId	获取实体 ID
setLookup	设置查找状态
isLookup	获取查找状态
getDataCount	获取数据数量
setFilterParameter	设置过滤参数
setListFields	设置列表字段
setSelectFields	设置选择字段
getProvider	获取提供者
setProvider	设置提供者
getPkFields	获取主键字段
setPkFields	设置主键字段
setFieldControlRules	设置字段控件规则
getQueryResult	获取查询结果
getRegisterPropertyListeners	获取注册属性监听
setRegisterPropertyListeners	设置注册属性监听
getSelectFields	获取选择字段

setSelectFields	设置选择字段
-----------------	--------

四、案例演示

案例 1：主实体模型

通过主实体模型获取单据头和单据体上的字段属性

```
public class MainEntityTypeSample extends AbstractFormPlugin {
    @Override
    public void afterBindData(EventObject e) {
        // 获取当前表单的主实体模型
        MainEntityType mainEntityType = this.getModel().getDataEntityType();
        // 基于表单主实体模型，创建空的数据包
        DynamicObject obj1 = new DynamicObject(mainEntityType);
        DynamicObject obj2 = (DynamicObject) mainEntityType.createInstance();
        // 获取主实体部分属性
        // 实体标识
        String entityNumber = mainEntityType.getName();
        // 标题，支持多语言
        LocaleString entityCaption = mainEntityType.getDisplayName();
        // 物理表名
        String tableName = mainEntityType.getAlias();
        // 应用标识
        String bizAppld = mainEntityType.getAppld();
        // 获取单据头上的字段属性
        IDataEntityProperty billNoProp1 = mainEntityType.getProperties().get("billno");
        IDataEntityProperty billNoProp2 = mainEntityType.getProperty("billno");
        IDataEntityProperty billNoProp3 = mainEntityType.findProperty("billno");
        // 获取单据体上的字段属性
        IDataEntityProperty beginDateProp1 = mainEntityType.getAllEntities().get("entryentity")
            .getProperty("qyrv_begindate");
        IDataEntityProperty beginDateProp2 = mainEntityType.findProperty("qyrv_begindate");
    }
}
```

案例 2：读取 DynamicObject 对象中存储的全部属性的值

本案例是逐层递归读取 DynamicObject 对象中存储的全部属性的值。

演示是需要注册表单插件，[从出差申请单列表中点击某条记录进行查看演示](#)

```
public class DynamicObjectSample extends AbstractFormPlugin {
    @Override
    public void afterBindData(EventObject e) {
        // 获取当前数据包
        DynamicObject dataEntity = this.getModel().getDataEntity();
        // 数据包是否从数据库加载，还是全新创建的？
        boolean isFromDB = dataEntity.getDataEntityState().getFromDatabase();
        // 简单属性，对应普通的字段，值是文本、数值、日期、布尔等
        String billno = (String) dataEntity.get("billno");
        // 复杂属性，关联引用的基础资料，值是另外一个数据包
        DynamicObject objTripType = (DynamicObject) dataEntity.get("qyrv_triptype");
        // 获取出差类型对应的基础资料中的出差类型的备注信息（注意要在单据中引用基础资料属性）
        String tripTypeName = (String) objTripType.get("qyrv_demo");
        // 集合属性，关联子实体，值是数据包集合
        DynamicObjectCollection rows =
dataEntity.getDynamicObjectCollection("entryentity");
        for (DynamicObject row : rows) {
            Date beginDate = (Date) row.get("qyrv_begindate");
            Date endDate = (Date) row.get("qyrv_enddate");
        }
    }
}
```